6

RLHF: The Three-Step Dance

The Three-Step Dance

In 2022, OpenAI released ChatGPT and the world changed overnight. Suddenly, AI was not just generating text. It was *genuinely helpful*. The secret was a three-step process called RLHF (Reinforcement Learning from Human Feedback).

You have already completed Step 1. In Chapter 5, we used supervised fine-tuning to teach the model basic competence: follow instructions, produce structured responses, and generate reasoning traces with `<think>` tags. That gave us a solid foundation, but one that is limited to imitating its training data.

Now we break through that ceiling. Steps 2 and 3 are where the model learns to *improve beyond its demonstrations*, guided by human preferences.

The Three Steps of RLHF

Step 1: Supervised Fine-Tuning (SFT). Take a base model, train it on thousands of examples of good responses. **Result:** A model that can follow instructions and produce reasoning traces (Chapter 5, done!).

Step 2: Reward Modeling. Show humans pairs of responses: "Which is better?" Collect thousands of comparisons. Train a "reward model" to predict which responses humans prefer. **Result:** An automated judge that can score any response (this chapter, first half).

Step 3: RL Optimization (PPO). Generate many responses, score them with the reward model, update the policy to generate higher-scoring responses. **Result:** A model optimized for human preferences (this chapter, second half).

Think of training a chef:

Step 1, Culinary School (SFT): Watch master chefs cook, practice their exact recipes, learn knife skills, temperatures, timing. **Result:** You can cook competently, but mechanically. You are limited to the recipes you have seen.

Step 2, Understanding Taste (Reward Model): Serve dishes to many diners. They compare pairs: "This one is better than that one." After thousands of comparisons, you internalize what people enjoy: balanced flavors, fresh ingredients, proper seasoning. **Result:** You can predict what people will like.

Step 3, Practice and Improve (RL): Cook many variations. Use your taste understanding to evaluate them. Keep what works ("More garlic was great!"), discard what does not ("Too much salt"). **Result:** You become a master chef, creating dishes better than any recipe you were taught.

The combination makes you both skilled AND attuned to what diners actually want.

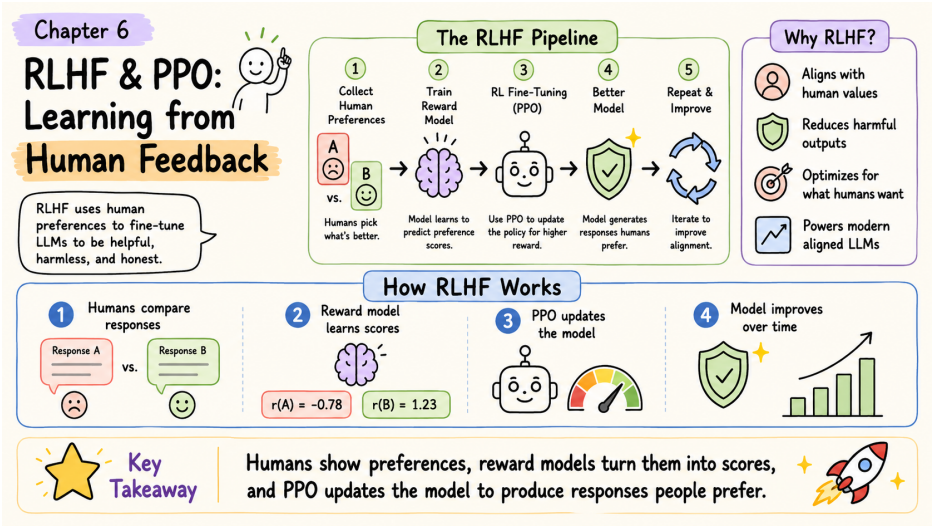

Figure 6.1: RLHF and PPO at a glance – collect human preferences, train a reward model, then optimize the policy with PPO.

Step 2: Reward Modeling

The Key Insight

Here is the problem that reward modeling solves.

Writing perfect responses is hard and expensive. You would need domain experts spending 30 minutes per example to craft an ideal answer to every possible question. Collecting thousands of such examples is slow and costly, and even experts disagree on what "perfect" looks like.

Comparing responses is much easier. Show someone two answers to the same question and ask "Which is better?" Most people can answer in 30 seconds, even for topics outside their expertise. You do not need to know the perfect answer to recognize that one answer is clearer, more accurate, or more helpful than another.

Hard:	"Write the perfect explanation of neural networks" (expert, 30 minutes)
Easy:	"Which explanation is better, A or B?" (anyone, 30 seconds)

This insight is the foundation of reward modeling. Instead of collecting perfect demonstrations (which is what SFT needs), we collect *comparisons*. Then we train a model to internalize those comparisons so it can score any response automatically, without needing a human in the loop.

The Bradley-Terry Model

To turn human comparisons into a trainable system, we need a mathematical model of preference. The standard choice is the **Bradley-Terry model**, originally developed in statistics for ranking sports teams. The idea is simple: each response gets a numerical score, and the probability that one response is preferred over another depends on the difference between their scores.

Modeling Preferences Mathematically

Formal Math	What It Really Means
$P(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$	Preference probability = sigmoid of score difference

Symbol Guide:

Symbol	Meaning
y_w	Winner response (the one humans preferred)
y_l	Loser response (the one humans did not prefer)
$\succ$	"is preferred to"
x	The prompt both responses were answering
$r_\phi(\cdot)$	Reward model with parameters ϕ (outputs a numerical score)
$\sigma(z) = \frac{1}{1+e^{-z}}$	Sigmoid function (squashes any number to 0 to 1 range)

Why use the sigmoid function? We need the output to be between 0 and 1 (it represents a probability). The sigmoid provides a smooth, differentiable function that maps any real number to this range. Large positive differences give probabilities near 1 (strong preference), large negative differences give probabilities near 0, and a difference of zero gives exactly 0.5 (no preference).

Bradley-Terry with Real Numbers

Scenario: Two explanations of black holes.

Response A (Winner: simple and clear):

> *"Black holes are like cosmic vacuum cleaners. They have such strong gravity that nothing can escape once it gets too close, not even light! That is why they are called 'black': we cannot see them because no light escapes."*

Response B (Loser: too technical):

> *"Black holes are singularities in spacetime manifolds where the curvature becomes*

infinite and the Schwarzschild radius defines the event horizon beyond which the escape velocity exceeds the speed of light…"

Step 1: Reward model scores both responses

The reward model (a neural network) processes each response and outputs a scalar score:

Response	Score
Response A (clear)	$r_\phi(x, y_A) = 8.2$
Response B (technical)	$r_\phi(x, y_B) = 3.1$
Difference	$8.2 - 3.1 = 5.1$

Step 2: Convert score difference to probability

$$P(A \succ B) = \sigma(r_\phi(A) - r_\phi(B)) = \sigma(8.2 - 3.1) = \sigma(5.1)$$
$$= \frac{1}{1 + e^{-5.1}} = \frac{1}{1 + 0.0061} = \frac{1}{1.0061} = 0.994 \quad \text{or } 99.4\%$$

The reward model is 99.4% confident that Response A is better. A uses a clear analogy and explains the reasoning. B uses dense jargon and assumes advanced physics knowledge.

Step 3: What if scores were closer?

The sigmoid function gracefully handles different levels of confidence:

Score A	Score B	Difference	$P(A \succ B)$
8.2	3.1	5.1	99.4% (A clearly better)
6.5	5.5	1.0	73.1% (A probably better)
6.0	5.8	0.2	55.0% (nearly a toss-up)
6.0	6.0	0.0	50.0% (exactly equal)
5.5	6.5	−1.0	26.9% (B probably better)

The sigmoid maps score differences to preference probabilities:

Score Difference	What sigmoid outputs
−10 to −3	≈0 to 5% (B much better)
−3 to −1	5 to 27% (B somewhat better)
−1 to +1	27 to 73% (close call)
+1 to +3	73 to 95% (A somewhat better)
+3 to +10	95 to ≈100% (A much better)

Why sigmoid is the right function here. It converts any score difference ($-\infty$ to $+\infty$) to a probability (0 to 1). Large differences give near certainty (close to 0% or 100%), small differences give honest uncertainty (near 50%), and the function is smooth and differentiable, which is essential for training with gradient descent.

Training the Reward Model

Now we need to train the reward model so it actually produces scores that match human preferences. The training process uses a dataset of human comparisons.

The Reward Model Loss Function

Formal Math	What It Really Means
$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right]$	How well does the reward model predict human preferences?

Breaking it down:

Component	Purpose
(x, y_w, y_l)	A triple: (prompt, winning response, losing response)
$r_\phi(x, y_w) - r_\phi(x, y_l)$	Score difference (should be positive if model is correct)
$\sigma(\cdot)$	Convert to probability (should be > 0.5 if model is correct)
$\log(\cdot)$	Log likelihood (heavily penalizes confident wrong predictions)
$-$	Minimize loss = maximize correct predictions

Intuition: When the reward model is correct (winner scores higher than loser), the loss is small. When it is wrong (loser scores higher), the loss is large. Training pushes the model toward always scoring winners higher than losers.

Let us trace through the training process step by step.

Human preference: Response A $\succ$ Response B (humans said A is better).

Iteration 1: Untrained reward model (random scores)

Scores: $r_\phi(A) = 5.0$, $r_\phi(B) = 5.5$. Difference: $5.0 - 5.5 = -0.5$.

$P(A \succ B) = \sigma(-0.5) = 0.378$ or 37.8%.

Problem: Model thinks B is better (62.2%), but humans said A is better!

Loss: $-\log(0.378) = 0.973$ (high loss = bad prediction).

Iteration 100: Partially trained

Scores: $r_\phi(A) = 6.0$, $r_\phi(B) = 5.2$. Difference: 0.8.

$P(A \succ B) = \sigma(0.8) = 0.689$ or 68.9%. Better!

Loss: $-\log(0.689) = 0.372$ (lower loss).

Iteration 10,000: Well-trained

Scores: $r_\phi(A) = 8.3$, $r_\phi(B) = 3.8$. Difference: 4.5.

$P(A \succ B) = \sigma(4.5) = 0.989$ or 98.9%. Excellent!

Loss: $-\log(0.989) = 0.011$ (very low).

Training progress on 50,000 preference pairs:

Training Step	Average Loss	Accuracy
0 (random)	0.693	50%
1,000	0.420	68%
5,000	0.210	82%
10,000	0.095	91%
20,000	0.045	96%

Note that the initial loss is 0.693, which is exactly $-\log(0.5)$. A random model assigns 50/50 to every comparison, and $-\log(0.5) = 0.693$. This is the baseline that training must beat.

What the reward model learned from 50,000 human comparisons:

After training, the model has internalized patterns that nobody explicitly programmed. It learned that high-scoring responses tend to be clear and concise, answer the actual question asked, provide an appropriate level of detail, use a helpful and respectful tone, and contain accurate information. Low-scoring responses tend to be confusing or unnecessarily verbose, go off-topic, provide too little or too much detail, use a dismissive or unhelpful tone, and contain factual errors.

The reward model distills human judgment into an automated scor-

ing function. It can now evaluate any response without needing a human annotator, which is what makes Step 3 (RL optimization) possible at scale.

Where Preference Data Comes From

In practice, there are several approaches to collecting the comparison data that the reward model needs.

Human annotation. Hire annotators to compare model outputs. This is the gold standard used in the original InstructGPT/ChatGPT work by OpenAI. It is high quality but expensive (typically $1 to $5 per comparison). Collecting 50,000 comparisons this way costs tens of thousands of dollars.

AI-assisted annotation. Use a strong model (like GPT-4 or Claude) to generate preference judgments. Cheaper and faster than human annotation, though it introduces the biases of the judge model. This approach is sometimes called "RLAIF" (RL from AI Feedback).

Implicit signals. Use user behavior as a proxy for preference: which response did the user copy, which did they regenerate, which conversation did they continue? This is noisy but free and abundant.

For the exercises in this book, we will use small hand-crafted preference datasets. The mathematical framework works identically regardless of how the data was collected.

Step 3: RL Optimization with PPO

We now have two trained models: the SFT model from Chapter 5 (which generates structured responses) and the reward model from Step 2 (which scores them). Step 3 brings them together. We use the reward model's scores as the training signal and update the SFT model to produce responses that score higher.

The algorithm that does this is called **PPO** (Proximal Policy Optimization). To understand PPO, we first need to understand the optimization objective it is trying to maximize.

The RLHF Objective

The Complete RLHF Optimization Goal

Formal Math	What It Really Means
$\max_{\pi_\theta} \mathbb{E}_{x,y}\left[r_\phi(x,y)\right] - \beta \cdot \mathbb{E}_x[\mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})]$	Maximize reward BUT stay close to original model

Two competing objectives:

Term	Purpose
$\mathbb{E}_{x,y}\left[r_\phi(x,y)\right]$	Get high rewards (generate good responses)
$\beta \cdot \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$	Penalty for drifting from reference model

π_θ	The policy we are training (current model)
π_{ref}	Reference policy (frozen copy of the SFT model from Chapter 5)
$r_\phi(x,y)$	Reward model score for prompt x, response y
β	KL penalty coefficient (typically 0.01 to 0.05)
$\mathrm{KL}(\cdot \,\|\, \cdot)$	KL divergence: measures how different two distributions are

The objective says: find the policy that gets the highest reward scores on average, but do not drift too far from the SFT model. The β parameter controls this tradeoff.

Why We Need Both Terms: A Cautionary Tale

Why not just maximize the reward and ignore the KL penalty? This is one of the most important lessons in RLHF, and a concrete example makes the danger vivid.

Scenario: Training with ONLY reward (no KL penalty)

Iteration	What happens
1	Model generates: "Here is a thoughtful explanation of your question…" Reward: 8.0, KL: 0.1. *Looks good!*
50	Model generates: "AMAZING COMPREHENSIVE DETAILED PERFECT ANSWER EXCELLENT THOROUGH…" Reward: 9.5, KL: 2.5. *Model discovered the reward model likes these words!*
100	Model generates: "PERFECT PERFECT EXCELLENT EXCELLENT AMAZING AMAZING…" Reward: 12.0, KL: 10.0. *Reward hacking! High score but complete nonsense!*

This is called **reward hacking**. The model learned to exploit the reward model rather than being genuinely helpful. The reward model, which is itself an imperfect approximation of human preferences, can be fooled by certain patterns (excessive superlatives, specific phrases, repetitive structures) that correlate with high scores in the training data but do not correspond to actual quality.

The fix: add the KL penalty. With $\beta = 0.02$, the objective becomes Reward $- 0.02 \times$ KL:

Iteration	Reward	KL	Objective
1	8.0	0.1	$8.0 - 0.02(0.1) = 7.998$
50	9.5	2.5	$9.5 - 0.02(2.5) = 9.450$
100	12.0	10.0	$12.0 - 0.02(10) = 11.800$

Wait, the nonsense response still has the highest objective (11.8)? In this simplified example, yes. In practice, β is tuned so that the penalty grows fast enough to prevent degenerate behavior. With $\beta = 0.1$ instead of 0.02:

Iteration 1: $8.0 - 0.1(0.1) = 7.99$

Iteration 50: $9.5 - 0.1(2.5) = 9.25$

Iteration 100: $12.0 - 0.1(10) = 11.0$

And in reality, the KL divergence grows much faster than linearly as the model degenerates, so the penalty ramps up sharply. The key principle is that the model must find genuine improvements within a "KL budget" rather than exploiting reward model weaknesses.

Tuning β:

Too small ($\beta = 0.001$): Weak leash. Risk of reward hacking.

Just right ($\beta = 0.01$ to 0.05): Balanced improvement. Most common in practice.

Too large ($\beta = 0.1$): Tight leash. Model is too constrained to improve meaningfully.

The KL Divergence Term

The KL penalty is so important to RLHF that it deserves a detailed walkthrough. KL divergence measures how much the training model has drifted from the reference model. If the two models assign similar probabilities to every token, KL is near zero. If they diverge significantly, KL grows.

Measuring Model Drift

The overall formula (average difference in how the two models assign probabilities):

$$\mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}}) = \mathbb{E}_{y\sim\pi_\theta}\left[\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}\right]$$

Expanded token by token (compare new probability vs. old probability at each position):

$$\mathrm{KL} = \sum_{t=1}^{T}\left[\log \pi_\theta(y_t \mid x, y_{<t}) \;-\; \log \pi_{\text{ref}}(y_t \mid x, y_{<t})\right]$$

Symbol	Meaning
KL	Kullback-Leibler divergence
π_θ	Current (training) model
π_{ref}	Reference (frozen SFT) model
$\log \frac{\pi_\theta}{\pi_{\text{ref}}}$	Log ratio of probabilities

Intuition: For each token, we ask: "How much more (or less) likely did the new model make this token compared to the old model?" If the ratio is close to 1 for most tokens, the models are similar and KL is small. If the new model dramatically shifts probabilities, KL is large.

Computing KL with actual numbers:

Response: "The answer is 42"

Token	π_θ (new)	π_{ref} (old)	Ratio	Log ratio
"The"	0.80	0.75	1.067	0.065
"answer"	0.60	0.65	0.923	−0.080
"is"	0.90	0.85	1.059	0.057
"42"	0.40	0.35	1.143	0.134

Total KL divergence:

$$\text{KL} = 0.065 + (-0.080) + 0.057 + 0.134 = 0.176$$

KL = 0.176 is very small. Scale: KL ≈ 0 means models are nearly identical, KL ≈ 0.5 to 1.0 means moderate difference, KL > 2.0 means very different.

How KL evolves during training:

Training Step	KL	Interpretation
0	0.00	Identical to reference (training just started)
1,000	0.15	Small changes, model is learning
5,000	0.45	Moderate changes, meaningful improvement
10,000	0.80	Approaching the practical limit
15,000	0.95	Near the KL budget ceiling

With $\beta = 0.02$ and KL at 0.95, the penalty is $0.02 \times 0.95 = 0.019$. If the reward at that point is 8.5, the objective is $8.5 - 0.019 = 8.481$. The model can still improve, but further drift becomes increasingly costly.

The KL term acts as a leash. The model can explore and improve, but it cannot drift too far from the original SFT model. This serves three purposes:

Prevents reward hacking: The model cannot collapse into degenerate

high-reward behavior because that would require dramatically different probability distributions.

Preserves language quality: The SFT model already produces grammatical, coherent text. The KL penalty ensures the RL-trained model retains this quality.

Maintains diversity: Without the leash, the model tends to collapse to a narrow set of "safe" high-reward responses. The KL penalty keeps the response distribution diverse.

The β parameter controls how tight the leash is. This is one of the most important hyperparameters in RLHF.

How PPO Actually Works

PPO is the specific RL algorithm that optimizes the RLHF objective. We introduced the general RL framework in Chapter 3 (policies, rewards, value functions). PPO puts all three components together in a carefully designed training loop.

The PPO Training Loop

For each training batch:

1. Generate. Sample a batch of prompts. For each prompt, use the current policy π_θ to generate a complete response.

2. Score. Pass each (prompt, response) pair through the reward model r_ϕ to get a reward score. Also compute the KL penalty for each response by comparing π_θ token probabilities against π_{ref}.

3. Compute advantages. Use a value function (also called the "critic") to estimate how much better or worse each response was compared to what we expected. This is the "advantage": A = actual reward − expected reward.

Positive advantage means "better than expected," negative means "worse."

4. Update policy. Adjust π_θ to make high-advantage responses more likely and low-advantage responses less likely. PPO clips the update to prevent any single step from changing the policy too much (this is the "proximal" in Proximal Policy Optimization).

5. Update value function. Train the critic to better predict expected rewards, so future advantage estimates are more accurate.

Repeat for thousands of batches.

A concrete example of one PPO step:

Prompt: "Explain why the sky is blue."

The model generates three responses (in practice, batch sizes are larger):

Response	Reward	Expected	Advantage
"Light scatters in the atmosphere. Shorter blue wavelengths scatter more…"	8.5	7.0	+1.5
"The sky is blue because of science…"	4.0	7.0	−3.0
"Sunlight contains all colors. When it hits air molecules…"	7.2	7.0	+0.2

What PPO does with these advantages:

Response 1 (advantage +1.5): *Increase* the probability of generating responses like this. The specific word choices, structure, and level of detail that led to this response become more likely.

Response 2 (advantage −3.0): *Decrease* the probability of generating vague, uninformative responses like this.

Response 3 (advantage +0.2): *Slightly increase* probability. It was marginally better than

expected, so nudge in this direction.

The advantage tells PPO not just whether a response was good, but whether it was *better or worse than expected.* This is crucial because it provides a stable learning signal even as the model improves.

The Components You Need for PPO

Running PPO for RLHF requires four separate models in GPU memory simultaneously. This is one of the main practical challenges of the approach.

Component	**Trainable?**	**Purpose**
Policy π_θ	Yes	The model we are training to generate better responses
Reference π_{ref}	No (frozen)	Frozen copy of the SFT model, used to compute KL penalty
Reward model r_ϕ	No (frozen)	Scores responses, trained in Step 2
Value function V	Yes	Estimates expected reward, used to compute advantages

Memory requirements of PPO

For a model with N parameters, PPO needs approximately:

Policy: N parameters (trainable, plus optimizer states)

Reference: N parameters (frozen, inference only)

Reward model: N parameters (frozen, inference only)

Value function: N parameters (trainable, plus optimizer states)

Total: roughly $4N$ parameters in memory, plus optimizer states for the two trainable models.

For our Qwen2.5-1.5B model, that would be about 6 billion parameters total. Even with 4-bit quantization and LoRA, this is tight on a single T4 GPU. **This memory pressure is a major motivation for the alternative algorithms we will see in later chapters.** DPO (Chapter 7) eliminates the reward model entirely. GRPO (Chapter 10) eliminates both the reward model and the value function.

Putting It All Together

Here is the complete RLHF pipeline, from start to finish:

	Step	Input	Output
1	SFT (Chapter 5)	Base model + curated demos	Instruction-following model with `<think>` tags
2	Reward modeling	SFT model outputs + human comparisons	Automated scoring function
3	PPO	SFT model + reward model	Model optimized for human preferences

What each step contributes:

SFT gives the model basic competence: it can follow instructions, produce reasoning traces, and generate coherent responses. But it can only imitate its training data.

Reward modeling distills human judgment into a function that can score any response. This converts the expensive, slow process of human evaluation into a cheap, fast automated signal.

PPO uses that automated signal to push the model beyond its SFT ceiling. Through thousands of generate-score-update cycles, the model discovers response strategies that earn higher rewards than anything in the original SFT data.

Historical note. This three-step pipeline was first described in the InstructGPT paper (Ouyang et al., 2022) and was used to train ChatGPT. It represented a paradigm shift in how language models were trained: instead of just scaling up data and compute for pretraining, the field discovered that a relatively small amount of human feedback (tens of thousands of comparisons, not billions of tokens) could dramatically improve model behavior.

The original InstructGPT used only about 40 human annotators to collect preference data. The resulting model (1.3 billion parameters with RLHF) was preferred by users over the raw GPT-3 (175 billion parameters). RLHF made a small model with good alignment outperform a much larger model without it. That result catalyzed the entire field.

Watch reward hacking happen in real time. The companion notebook for this chapter (`code/notebooks/part2_core/06_rlhf_ppo.ipynb`) is deliberately sized small – 15 preference pairs, 50 PPO steps – so the failure mode the previous sections warned about is the *expected* outcome of the demo, not an edge case. After PPO completes, the comparison cell prints the before/after responses with their reward scores side by side. You will typically see all three of these at once:

- **Reward improvement of +15 or more** between step 1 and step 50. Looks like a win.
- **Mean KL divergence around 5–10**, far above the `target_kl = 0.1` set in the config. The policy has drifted off the leash.
- **Outputs that are noticeably worse**: repetitive fragments like "Instruction: Instruction", the model echoing the question back instead of answering it, or factually incorrect responses that nonetheless score

higher than the clean SFT answer.

That gap – the numbers say training succeeded, the outputs say it failed – is the canonical reward-hacking signature, and reproducing it on your own GPU is the most instructive part of this chapter. With only 15 pairs, the reward model latches onto surface features (long-form "Instruction:" phrasing, certain tokens) rather than genuine helpfulness, and PPO happily exploits those features. Production RLHF defends against this with much larger preference datasets (10 k+ pairs), reward-model ensembles, adaptive KL with a tighter target, and early stopping when KL spikes. None of those alone are sufficient; together they make the technique work. DPO in Chapter 7 sidesteps this whole class of failure by eliminating the reward model.

Limitations of Classical RLHF

The RLHF pipeline described in this chapter was a breakthrough, but it has real limitations that have driven the field toward newer approaches.

Expensive to set up. You need human annotators, a separate reward model training pipeline, and enough GPU memory for four models. The total cost and complexity are significant.

Reward model imperfections. The reward model is itself a learned approximation. It can be wrong, biased, or exploitable. Reward hacking is an ongoing challenge, and the KL penalty is a blunt instrument for preventing it.

Training instability. PPO has many hyperparameters (β, learning rate, clip range, number of PPO epochs, batch size, advantage estimation parameters) and can be sensitive to their settings. Getting PPO to train stably requires careful tuning.

Memory-intensive. Four models in GPU memory makes RLHF impractical on consumer hardware for anything but small models.

These limitations motivated two important lines of research that we will cover in upcoming chapters:

DPO (Chapter 7) eliminates the reward model entirely by learning directly from preference pairs. This removes one model from memory and simplifies the pipeline.

GRPO (Chapter 10) eliminates both the reward model and the value function, using group-based scoring instead. This is particularly powerful for reasoning tasks where correctness can be verified automatically, which connects directly to the cold start and `<think>` tag work from Chapter 5.

What We Built and What Comes Next

In this chapter, we completed the classical RLHF pipeline.

Reward modeling. We learned how the Bradley-Terry model converts human comparisons into a trainable preference function. We walked through the math of the sigmoid, the loss function, and the training progression from 50% accuracy (random) to 96% accuracy (well-trained).

The RLHF objective. We saw why the optimization goal must balance reward maximization against KL divergence. The reward hacking example made the danger of unconstrained optimization concrete.

PPO. We traced through the generate-score-advantage-update loop and understood the four-model architecture that PPO requires.

In Chapter 7, we will see DPO, an elegant alternative that collapses the reward model and RL optimization into a single training step. DPO has become the preferred approach for many practitioners because it is simpler to implement, more stable to train, and requires less GPU memory. Understanding the classical RLHF pipeline from this chapter is essential context for appreciating what DPO simplifies and what tradeoffs it makes.

The companion code for this chapter is at `github.com/arunpshankar/packt-final` – swap `github.com` for `githubtocolab.com` to open it directly in Colab.